Clinical Decision Support System (CDSS)

Using Machine Learning for Heart and Diabetes Risk Prediction

A Project Report

Submitted by:

Garv Patel - (AU2240251)

in partial fulfillment for the award of the degree

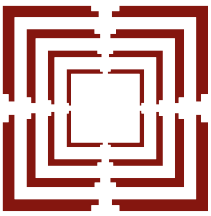
of

BACHELOR OF TECHNOLOGY

in

Computer Science and Engineering

at



Ahmedabad University

School of Engineering and Applied Science (SEAS)

Ahmedabad, Gujarat

May, 2026

DECLARATION

I hereby declare that the project entitled "**Clinical Decision Support System (CDSS)**" submitted for the B. Tech. (**CSE**) degree is my original work and the project has not formed the basis for the award of any other degree, diploma, fellowship or any other similar titles.

Signature of Student

Date:

Place:

CERTIFICATE

This is to certify that the project titled "**Clinical Decision Support System (CDSS)**" is the bona fide work carried out by **Garv Ketankumar Patel**, a student of B. Tech. (**CSE**) of School of Engineering and Applied Science at Ahmedabad University during the academic year 2025-2026, in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in **Computer Science and Engineering** and that the project has not formed the basis for the award previously of any other degree, diploma, fellowship or any other similar title.

This project was done under the supervision of the faculty mentor **Dr. Sanjay Chaudhary**.

Signature of Faculty Mentor

Date:

Place:

Abstract

Cardiovascular disease and diabetes are the world's most prevalent and lethal chronic diseases but early detection is complicated by a lack of diagnostic skills. In order to assist doctors in their diagnostic processes, this project outlines the creation and setting up of a system that leverages learning techniques to assess the threat of diabetes and cardiovascular diseases.

With the intention of providing a probability-based risk score and categorize the risk into three risk levels—Low, Medium, and High—with medical recommendations, the system uses a Random Forest Classifier that was separately trained on Heart Disease Dataset and Diabetes Dataset. The machine learning models are monitored using MLflow to ensure systematic tracking and management of the models.

The inference engine is deployed as a **RESTful API** with **FastAPI**, containerised with **Docker** and **Docker Compose** for consistency and portability. Real-time monitoring is provided by **Prometheus** and **Grafana** for data collection and dashboard display, respectively, to create a **MLOps** framework from model development to production monitoring. The system illustrates the viability of operationalising machine learning in production-grade, scalable healthcare decision support systems.

Table of Contents

Declaration i
Certificate ii
Abstract iii
Table of Contents iv

List of Figures **vi**

List of Tables **vii**

Gantt Chart **viii**

1 Introduction **1**
1.1 Project Definition 1
1.2 Project Objectives 2

2 Literature Survey **4**
2.1 Related Work 4
2.2 Tools and Technologies 5
2.2.1 Python 5
2.2.2 Scikit-learn 5
2.2.3 Random Forest Classifier 6
2.2.4 FastAPI 6
2.2.5 Docker 7
2.2.6 MLflow 7
2.2.7 Prometheus 7
2.2.8 Grafana 7
2.2.9 Git 7

3 System Design and Methodology **8**
3.1 System Architecture 8
3.2 Data Gathering 9
3.3 Data Preprocessing 9
3.4 Training of Model 10
3.5 Model Assessment 11
3.6 Model Deployment 13
3.7 Monitoring System 13
3.8 Prediction Workflow 14

4 Results **16**
4.1 Project Outcomes 16
4.2 My Contributions to the Project 18
4.3 Learning Outcomes 19
4.4 Real World Applications 19

5 Conclusion **21**

Bibliography **21**

List of Figures

0.0.1 Gantt Chart viii

2.2.1 Model Details - Random Forest Classifier. 6

3.1.1 System Architecture 9
3.5.1 Confusion Matrix — for Heart Disease model and Diabetes model. 12
3.5.2 Performance Evaluation Metrics Based on Confusion Matrix 12
3.8.1 Prediction Workflow Flowchart. 14
3.8.2 Sequence diagram of the CDSS. 15

4.1.1 Website Prediction Interface for Heart Disease prediction. 17
4.1.2 Website Prediction Interface for Diabetes prediction. 17
4.1.3 Grafana Monitoring Dashboard — showing real-time panels. 18

List of Tables

3.2.1 Dataset Summary. 9
3.5.1 Risk Stratification Thresholds. 11
3.6.1 Docker Services. 13

4.1.1 Model Performance Summary. 17

Gantt Chart

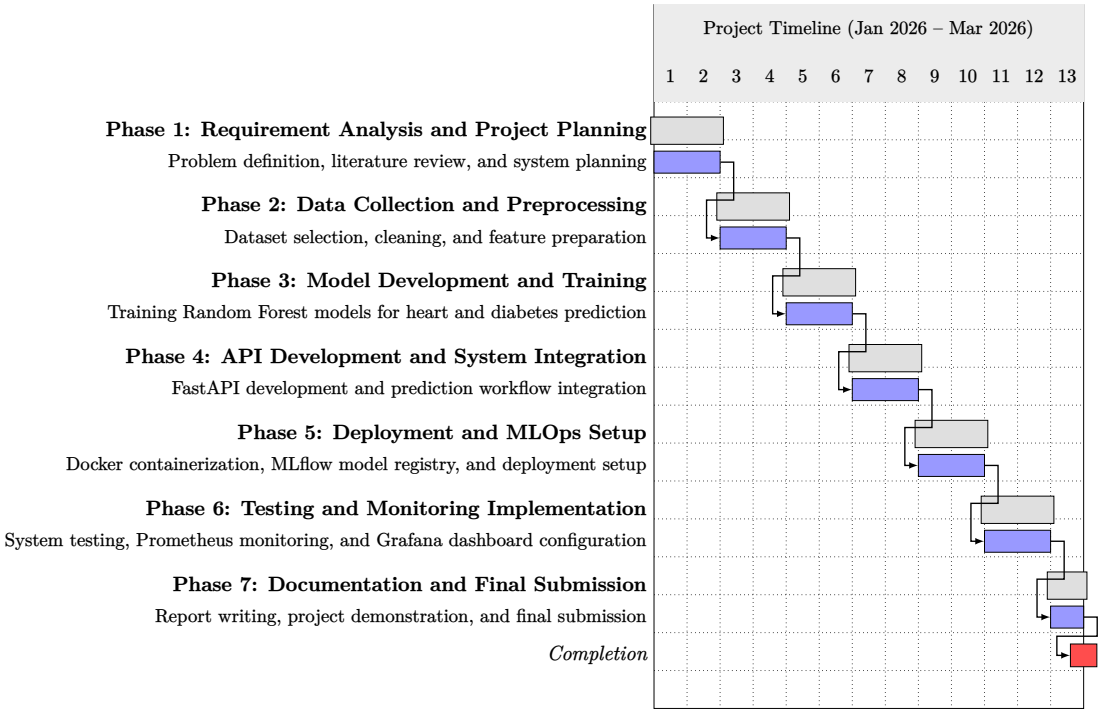


Figure 0.0.1: Gantt Chart

Chapter 1

Introduction

Healthcare can now be better due to developments in technologies. The creation of an intelligent system to help medical professionals make prompt and precise judgments is one example. Here, we describe the creation of a Clinical Decision Support System (CDSS) that uses learning algorithms to forecast the threat of diabetes and heart disease, mainly considered as most common chronic illnesses. A strong MLOps approach was used in the system's construction to enable efficient deployment, tracking, and monitoring in real-world scenarios.

1.1 | Project Definition

Cardiovascular disease and diabetes are among the many non-communicable diseases (NCDs) dominating the world's healthcare. Accurate and timely diagnosis is essential in medical practice, but traditional diagnostic processes often rely heavily on physician interpretation, manual data entry and are often laboratory-based and time-consuming processes. This dependency leads to delays, inconsistencies and accessibility problems - particularly in low-resource health-care facilities where specialist expertise may not be readily available. In this context, the importance of smart automated clinical systems is not only interesting, but critical.
This project is a health information technology system designed to assist health-care providers, such as physicians and clinicians, to make clinical decisions by providing patient-specific information and recommendations. A CDSS, unlike traditional diagnosis, uses data-driven analysis of patient data, biomarkers and previous health measurements to provide information, which eases the burden on doctors and clinicians from information overload and the potential for human error. Machine learning-driven CDSSs can discover non-linear relationships and interactions among features that may not be obvious from conventional approaches, and are good for risk scoring and prediction.
Cardiovascular disease and diabetes are two of the world's most prevalent and morbid diseases, and are the subject of this project. As reported by the World Health Organization (WHO), cardiovascular disease leads global mortality rates, resulting in approximately 17.9 million deaths each year. Additionally, Diabetes mellitus impacts over 422 million individuals worldwide; it stands not only as a significant health issue on its own but also serves as a critical risk factor for other serious conditions including heart disease, kidney failure, and various neuropathies. The intersection of these diseases also complicates the care of patients and indicates the need for "smart" prediction models.
Although this burden of disease has significant public health consequences, early prediction

of heart disease and diabetes is challenging. The symptoms of early stages of the disease are either absent or not specific and patients may only seek medical attention when the disease has progressed. And the complexity of the information (demographic, clinical, biochemical, lifestyle factors) collected about a patient means that manual disease prediction is unlikely to be thorough and reliable. This highlights a gap in the current health system that can be addressed by technology.
The application of machine learning techniques has brought a new wave of analytics in health care with robust predictive modelling in diagnosis, prognosis and treatment. Supervised learning techniques have proven promising in predicting disease classification, by learning from past patients' records to find discriminative patterns. The Random Forest classification approach, which mixes several decision trees to facilitate classification, is used in this project. This technique is well suited to medical data since it is resistant to overfitting, can deal with missing data, and is also interpretable using feature importance analysis - an important consideration in health care systems where it is important to explain the reasoning behind predictions.
The system in this project accepts structured data as input from the patient, passes it on to the machine learning models, and provides a three-tiered risk prediction of **"Low Risk"**, **"Medium Risk"** and **"High Risk"**. This classification provides prioritised health advice rather than a binary yes or no, therefore increasing the value of the predictions. The system also includes a MLOps system as part of the predictive modelling system - based on FastAPI, Docker, with MLflow tracking and Prometheus/Grafana monitoring - to provide a scalable, efficient and production-grade clinical decision support system.

1.2 | Project Objectives

The purpose of this project is to create a Support System that will be able to predict the risk of getting certain diseases like cardiovascular disease and diabetes using structured data. To attain this objective, we have defined the following objectives and diligently addressed them in the project.

1. The first objective is to **train model** for the classification of diabetes risk and heart disease. This process starts with data collecting and cleaning up, and progresses to exploratory data analysis, feature engineering and machine learning with Random Forest classifiers based on the classification performance measures of accuracy, precision, F1-score, recall, and AUCROC.

2. Using probability values produced by the trained models, create a **three-level risk stratification system** to classify patient outcomes into Low Risk, Medium Risk, or High Risk. This will provide clinically relevant results that allow for customized healthcare strategies according to the anticipated level of risk.

3. **Provide the trained models as a prediction service via FastAPI**, exposing RESTful endpoints that accept the feature inputs of patients and return the real-time classification of the risk and confidence, with a fast response time and ease of integration into user-friendly healthcare platforms.

4. **Containerise the entire application stack with Docker** to ensure that the deployment of all system components, be they the prediction API or monitoring tools,

is seamless, reproducible and environment-independent — with Docker Compose used to define the multi-container system for ease of deployment and maintenance.

5. **Incorporate ML experimentation tracking and model versioning with MLflow**, where training hyperparameters, model evaluation metrics and serialised model artefacts are tracked across the course of each experiment to ensure experiment reproducibility and model selection in the machine learning development cycle.

6. Sixth objective for this project is to set-up prometheus and grafana dashboard for **real-time monitoring** of the system.

7. **Provide a fully integrated and end-to-end MLOps system** that brings together all of the above components in a maintainable and scalable system design, showcasing the principles of production-grade machine learning engineering in the context of clinical healthcare analytics.

Chapter 2

Literature Survey

The work on developing clinical decision support systems that use machine learning builds on research in the areas of medical informatics, predictive modelling and artificial intelligence. This chapter reviews the relevant literature on disease prediction, history of clinical decision support systems and using machine learning to diagnosis disease. Furthermore, this chapter provides an overview of the technical frameworks employed during the system's development along with rationales for their selection based on contemporary research findings and industry benchmarks.

2.1 | Related Work

Over recent decades there has been extensive research focused on applying machine learning methodologies within healthcare settings. Large electronic health records (EHRs), digital health data sources, and massive computing capacity have made it possible to create data-based models that can help doctors diagnose, treat, and forecast a variety of medical diseases. Specifically, cardiovascular disease and diabetes mellitus have received a significant amount of attention due to their prevalence, complexity and the serious implications of a delayed diagnosis.

Historically, diagnostic approaches for heart disease and diabetes have involved a combination of physical examination, laboratory tests, imaging and clinical judgement. Although medically accurate, these methods are subject to a number of constraints. Manual diagnosis is laborious, costly and prone to inter-clinician variation — a situation in which one clinician may draw different conclusions from another when interpreting the same set of data. Furthermore, in areas with limited access to experts, patients often present with advanced disease, reducing the potential for successful treatment. Such shortcomings of traditional diagnostic processes highlight the need for objective, automated and scalable methods.

Studies of Clinical Decision Support Systems (CDSS) began in the early 1970s with pioneering systems like MYCIN, a rule-based expert system for bacterial infection diagnosis, proving the potential for computerised medical decision-making. But these rule-based CDSS were limited in their capacity to capture an external, comprehensive, and hand-coded knowledge base, and their capacity to extrapolate from pre-coded rules. The rise of machine learning heralded a shift towards systems that can learn rules implicitly from patient data rather than being explicitly programmed with domain knowledge.

A range of studies have shown the success of supervised learning approaches in disease classification. Detrano et al. pioneered early work in the automated diagnosis of coronary

artery disease (CAD) on the Cleveland Heart Disease Dataset, which has been widely adopted as a benchmark dataset for CAD classification tasks. More recent studies by Aha and Kibler (1991) and, more recently, by Mohan et al. (2019) have shown that ensemble classifiers, in particular Random Forest, consistently outperform individual classifiers on heart disease data due to their variance reduction effect and insensitivity to feature noise. The idea of risk stratification — grouping patients into ordered risk categories rather than a binary diagnosis — has emerged as a more useful approach. Goldstein et al. (2017) proposed that risk scores in the form of a risk probability, stratified into low, medium and high risk, greatly improve clinical utility by facilitating prioritised resource allocation and avoiding over- and under-treatment. This tripartite approach underpins the prediction model in this project.

While the major significance of research on the value of the learning models has been on their predictive performance, over the last few years, there has been a greater emphasis on the practice of using machine learning models in healthcare domain. MLOps (machine learning operations) address the challenges of model reproduction, version control, scaling and monitoring the performance of these models. Moving from development to deployment is a crucial, but frequently overlooked, engineering difficulty, according to Sculley et al. (2015) in their important paper on "hidden technical debt in machine learning systems." More recently, Kreuzberger et al. (2022) described the MLOps lifecycle, demonstrating that experiment management, containerisation and monitoring are key components of successful machine learning systems in production. The current project addresses these concerns by using MLflow, Docker, Prometheus and Grafana to implement an MLOps approach.

The challenges highlighted in the literature reviewed — the excessive reliance on single model systems, the lack of focus on deployment and monitoring, and the lack of risk stratification in many systems — have directly shaped the proposed CDSS. The integration of ensemble learning with an MLOps pipeline and a three-layered risk stratification approach aims to move beyond proof-of-concept systems and provide a system that is both accurate and practical for practical use in healthcare settings.

2.2 | Tools and Technologies

This section describes the technologies which are used in the designing, in development and in the deployment this System. For each of these, an explanation of the type of tool, the rationale for its use and how it was used in the project is provided.

2.2.1 | Python

Python serves as the foundational programming language throughout all phases: pre-processing data sets , model training , API creation ,and pipeline orchestration . Its compatibility with leading-edge machine-learning frameworks alongside web technologies positions it favorably for constructing integrated ML systems tailored towards healthcare solutions.

2.2.2 | Scikit-learn

Scikit-learn functions as an open-source library dedicated to providing efficient implementations encompassing several well-known algorithms used in machine learning via

Python modules featuring uniform access patterns across these algorithms. It provides a unified, documented API for estimating, cross-validating, tuning (using GridSearchCV) and evaluating models with measures including accuracy score, classification report and ROC-AUC score. In this example, Scikit-learn offers the RandomForestClassifier, train-testsplit, crossvalscore, GridSearchCV, StandardScaler and classificationreport functions used when training the model. Its predictproba() is used in particular to produce the continuous probability scores that are the basis for the three-step risk stratification in this project.

2.2.3 | Random Forest Classifier

In order to create a final prediction, Random Forest, a supervised ensemble machine learning technique, builds a set of decision trees during training and aggregates each tree's predictions (majority vote in classification). Random Forest uses two sources of randomness (in the sampling of the training data, called bootstrap aggregation or bagging, and in the features used to make decisions) to make a model less prone to variance, without being too biased. Random Forest was selected for this project because it has been successfully applied to tabular clinical data, it is naturally robust to overfitting and it can provide feature importance scores which are easy to interpret.

Parameter	Value	Why
n_estimators	*500*	*500 decision trees — more trees = smoother, more reliable probability estimates*
max_depth	10	Each tree can go 10 levels deep — captures complex patterns without overfitting
min_samples_split	3	A node must have at least 3 samples to split — reduces noise
min_samples_leaf	2	Each leaf must have at least 2 samples — prevents overfitting on outliers
class_weight	balanced	Automatically adjusts for imbalanced classes (e.g. more healthy than sick patients)
random_state	42	Fixes randomness — ensures reproducible results every run

Figure 2.2.1: Model Details - Random Forest Classifier.

2.2.4 | FastAPI

FastAPI is a brand-new, quick (high-performance) web framework that uses common Python type hints to create APIs with Python 3.6+. It is asynchronous and has one of the fastest Python frameworks. In this article, we use FastAPI to create REST APIs for predicting heart disease risk classification and diabetes risk classification. The API will receive a patient's features as a JSON, and will predict the patient's risk classification (1-4) and its probability using the pre-trained machine learning models.

2.2.5 | Docker

Applications and their dependencies are bundled into containers using Docker, an open source platform for containerizing software. Containers share the host operating system, are much lighter than virtual machines and provide the same level of process isolation. Here, we containerise the FastAPI prediction service, the MLflow server and the Prometheus and Grafana stack with Docker. We start and connect multiple containers in a multi-service deployment using a Docker Compose file, making deployment from development to production.

2.2.6 | MLflow

An open-source machine learning platform for managing, packing, and registering models and their cersions is called MLflow. In this project, MLflow is applied to record all training runs for the heart disease and diabetes models, store serialised model artefacts and register model versions for production in the MLflow Model Registry, enabling auditable model promotion.

2.2.7 | Prometheus

Prometheus is an open-source monitoring system and service by SoundCloud.. It make use of a ”pull” approach for metrics; metrics are periodically scraped from instrumented application endpoints, collecting time-series metrics of the application’s performance. In our project, we use Prometheus to instrument the FastAPI application using the **prometheus-fastapi-instrumentator** package to collect the run-time metrics, including API request rates, request latencies and prediction endpoint responses, to build the observability layer of our deployed system.

2.2.8 | Grafana

Grafana is available as open-source application that can create real-time dashboards and alarm management consoles by connecting to a variety of data sources, including Prometheus. For this project, Grafana is set up to access the Prometheus metrics and visualize on dashboard.
This dashboard gives operational users a real-time view of the deployed CDSS’ performance and health.

2.2.9 | Git

Git is a version control system (VCS) that monitors modifications made to a project’s source code. It simplifies work among multiple developers on the same codebase, keeps track of changes to the source code and supports branching and merging capabilities to support parallel development activities. In this project, Git is used for version control of the source code, configuration, training scripts, and build/pipeline scripts, tracking all changes made and enabling the project to be restored to any previous state.

Chapter 3

System Design and Methodology

This section contains the system design and method which helps to develop, design and deploy the Clinical Decision Support System. It includes the system design, data gathering and processing, model development and testing, API deployment, containerisation, and system monitoring. The process adheres to a machine learning engineering process, which is reliable, accurate and reproducible.

3.1 | System Architecture

The CDSS is a multi-tiered, containerised web application. It consists of seven main components that collaborate to provide real time disease risk assessment.
The **Frontend** is a web page that enables the user to enter patient health information and see a risk profile. It communicates with the Backend via HTTP POST calls with JSON patient data.
The **FastAPI Backend** is the main server. It provides prediction endpoint services for heart disease and diabetes, processes user inputs (validates and transforms), loads the trained models, and returns structured classification responses.
The **Machine Learning Models** — two separately trained Random Forest classifiers — are loaded during application initialisation and used to make predictions. The MLflow Model Registry provides versioning of models, tracking training metrics, hyperparameters and serialised model artifacts in a central store.
The **Docker Deployment** layer provides individual service containerisation. Docker Compose binds together the entire multi-service stack, defining service networking, port bindings and volume mounts in a declarative manner. The Prometheus service collects metrics from the API endpoint every 15 seconds, which is then visualised by Grafana on a live operations dashboard.

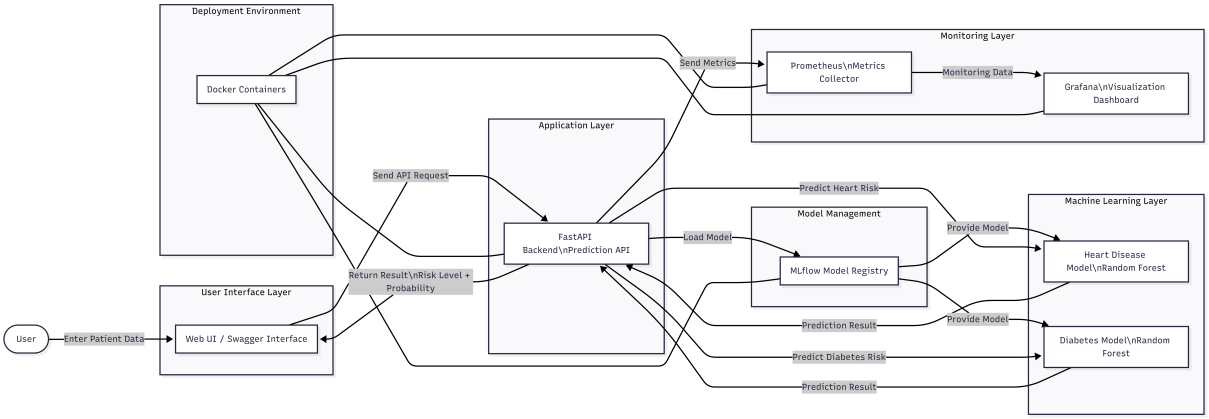


Figure 3.1.1: System Architecture

3.2 | Data Gathering

Two publicly accessible, clinically examined data sets are used to train the prediction models.
First dataset of heart disease which is **Cleveland Heart disease dataset** from UCI Machine Learning repository. It contains attributes like age, sex, type of chest pain, resting BP, serum cholesterol, fasting sugar level moreover ECG results, maximum heart rate along with ST depression, no. of vessels and thalassemia.
Second dataset is **Diabetes dataset** which conatins record of 768 patients. This dataset is taken from National Institute of Diabetes and Digestive and Kidney Diseases(NIDDK), available on kaggle. It contains 8 attributes such as age, body mass index, blood pressure, tricep skin tickness, glucose concentration, number of pregnancies for women, 2-Hour serum insulin.

Table 3.2.1: Dataset Summary.

Attribute	Heart Disease Dataset	Diabetes Dataset
Source	UCI ML Repository	NIDDK / Kaggle
Total Records	303	768
Number of Features	13	8
Positive Cases	165 (54.5%)	268 (34.9%)
Negative Cases	138 (45.5%)	500 (65.1%)
Target Variable	Heart disease (0/1)	Diabetic (0/1)

3.3 | Data Preprocessing

Medical data sets have quality problems that need to be addressed prior to model training. Both datasets undergo the same preprocessing.
Handling Missing Values: The Diabetes dataset includes physiologically implausible zero values in features like Glucose, BloodPressure, SkinThickness, Insulin and BMI, indicating missing data. They are imputed with column-wise median as the distributions of these features are skewed and mean imputation would systematically bias the results.

Data Cleaning: Duplicate instances are dropped from both datasets. Data types of features are explicitly set to ensure data type consistency when performing array calculations, and integer-encoded categorical features are checked for valid ranges.
Feature Selection: All features in both training and test sets are used for training as they are known to be important for clinical applications. The Random Forest model's feature importance after training is used to validate the predictive value of features.
Data Normalisation: Using **StandardScaler**, all features are normalised to zero mean and unit variance to prevent high magnitude features from having a stronger influence on the model. To avoid information leakage while scaling data processes for machine learning models' training sets—and subsequently applying these to both training and testing datasets—the scaler is first put to the training set alone. we use 80-20 ratio for training and test sets respectively to maintain class distribution integrity.

3.4 | Training of Model

The **Random-Forest-Classifier** is used in both illness prediction models. This supervised ensemble methodology employs random subsets of features at each split stage while constructing multiple decision trees derived from randomly chosen segments within the training data. It fits many decision trees and generates predictions by combining each decision tree's predictions using the mean for probability or the majority vote for classification. The ensemble nature of the algorithm reduces the variance of predictions, and offers a robust and generalisable predictor that works well with tabularised clinical data.
Train-Test Split: Stratified sampling methods ensure that classes are equally represented across both training and testing datasets for diabetes-related data exhibiting class imbalance—with an allocation reflecting 80% for training purposes against 20% reserved for testing outcomes.
Training: Hyperparameters—including **n_estimators** as well as **max_depth** alongside **min_samples_split**—are optimized through **GridSearchCV** utilizing five-fold stratified cross-validation where weighted F1 score serves as the target metric. Each model incorporates a **"balanced"** argument assigned to class weights addressing issues surrounding imbalanced classifications effectively.
Model Saving: Once training is complete, the **RandomForestClassifier** and its associated **StandardScaler** are saved as **.pkl** binary files using the Python library **joblib**, and are loaded by the FastAPI web service as needed.
Training Tracking with MLflow: Training is tracked in an MLflow experiment. Training parameters are logged with `mlflow.log_params()`, training metrics with `mlflow.log_metrics()` and models with `mlflow.sklearn.log_model()`. A sustainable model lifecycle has been established through registering optimal models into MLflow Model Registry followed by advancing them into production phases thereafter.

3.5 | Model Assessment

Using the standard metrics for classification, we calculate the model's performance. **Accuracy** represents the proportionate count of correct predictions made whereas **precision** indicates how many true positive cases exist among all predicted values—which highlights occurrences classified incorrectly. **Recall (Sensitivity)** is the number of true positives predicted - the most medically-significant measure as it's the worst case not to detect patients (false negative). **F1-Score** (harmonic mean of precision and recall) is balanced - "tune on this". The ability of the model to discriminate (not dependent on threshold) is the idea behind **AUC-ROC.**
Confusion Matrix provides insights regarding various types associated with errors throughout two specific risk prediction tasks categorizing every forecasted outcome into True positives, negatives and False postivies, negatives - TPs, TNs, FPs, FNs respectively.

Table 3.5.1: Risk Stratification Thresholds.

Predicted Probability (p)	Risk Tier	Clinical Recommendation
$p < 0.40$	Low Risk	Routine health monitoring; no immediate intervention required
$0.40 \leq p < 0.70$	Medium Risk	Lifestyle intervention advised; follow-up screening recommended
$p \geq 0.70$	High Risk	Immediate clinical consultation strongly recommended

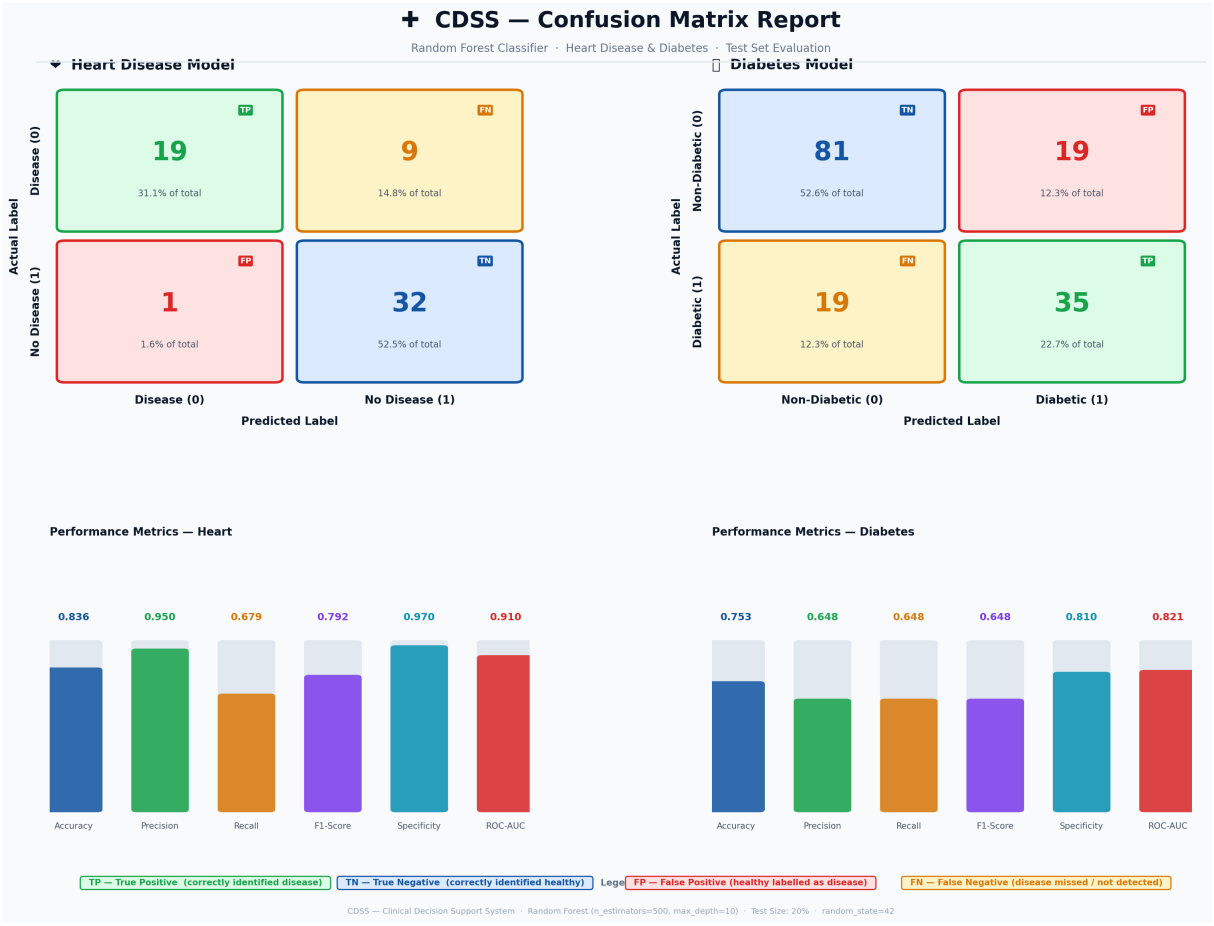


Figure 3.5.1: Confusion Matrix — for Heart Disease model and Diabetes model.

Metric	Formula	Description
Accuracy	Accuracy = (TP + TN) / (TP + TN + FP +FN)	Measures the overall correctness of the model predictions.
Precision	Precision = TP / (TP + FP)	Measures how many predicted positive cases are actually correct.
Recall (Sensitivity)	Recall = TP / (TP + FN)	Measures the ability of the model to correctly identify actual positive cases.
F1 Score	F1 = 2 × (Precision × Recall) / (Precision + Recall)	Represents the balance between Precision and Recall.
AUC–ROC	Area under the ROC curve (TPR vs FPR)	Measures the model's ability to distinguish between positive and negative classes.

Figure 3.5.2: Performance Evaluation Metrics Based on Confusion Matrix

3.6 | Model Deployment

FastAPI API Deployment: The models are deployed in a production REST API using FastAPI. The app exposes two prediction services — `/predict/heart` or `/predict/diabetes` — which expect a JSON request body that conforms to a Pydantic model. Once a valid request is received, the endpoint scales the data using the pre-fitted **StandardScaler**, makes a prediction with the respective Random Forest classifier, maps the probability to a risk stage, and returns a JSON response with the predicted risk, probability and a clinical recommendation.
Docker Containerisation: The FastAPI application is containerised with a **Dockerfile** that extends a `python:3.10-slim` base image, with all of the required Python packages installed from a **requirements.txt** file, and serialised model artefacts included in the image build. Uvicorn ASGI server is used to serve the application.
Docker Compose Usage: A **docker-compose.yml** file is used to define and manage the four services that constitute the deployment stack, mapping ports, mounting persistent volumes, setting up networking using an internal bridge network, and injecting environment variables. The entire system is started with `docker compose up`.

Table 3.6.1: Docker Services.

Service	Port	Purpose
`cdss-api`	8000	FastAPI prediction service
`mlflow-server`	5000	Experiment tracking and model registry UI
`prometheus`	9090	Metrics collection and time-series storage
`grafana`	3000	Real-time monitoring dashboard

3.7 | Monitoring System

The monitoring system offers real-time operational insights into the operational state and performance of the deployed CDSS. The FastAPI application is instrumented with the `prometheus-fastapi-instrumentator` library, which is automatically served from the `/metrics` endpoint scraped every 15 seconds by Prometheus.
Prometheus Metrics Collected: Metrics include the total number of HTTP requests grouped by endpoint path, method and status code; a histogram of request duration to support the calculation of p50, p90 and p99 percentiles of response latency; and a counter for the number of prediction events for each model at the heart disease and diabetes endpoint.
Grafana Dashboard Visualisation: Grafana is pre-configured with a Prometheus data source and dashboard with panels for real-time API request counts, response latencies at the 50th, 90th and 99th percentiles, prediction counts for each model, and error counts.
Request Counting and Latency Monitoring: Requests are automatically counted and labelled, allowing traffic anomalies to be detected. Increased p99 latencies are early indicators of performance degradation or bottlenecks.

3.8 | Prediction Workflow

The prediction process of the CDSS follows the following steps.
User Input: The end user inputs patient data through the frontend web form. The form is validated on the client side for completeness.
API Request: The client-side form data is serialised as a JSON object and sent via HTTP POST to the appropriate FastAPI prediction API endpoint.
Model Prediction: The API validates and extracts the feature vector from the payload against the Pydantic model input schema, transports the vector through the pre-fitted **StandardScaler** function to normalise the feature vector and feeds the scaled feature vector to the loaded Random Forest model to obtain a predicted class label and probability score via `predict_proba()`.
Risk Stratification: The probability is compared to the risk stratification criteria (Table 3.5.1) to determine the patient's clinical risk — Low, Medium, or High Risk.
Response Generation: The risk tier, probability, disease type and clinical recommendation are serialised as a JSON response and sent to the frontend with an HTTP 200 status code to be displayed to the end user.
Monitoring: Prometheus will automatically increment the number of requests and record the round-trip Time-to-Prediction on the next scrape, updating the Grafana dashboard within 15 seconds of a prediction request.

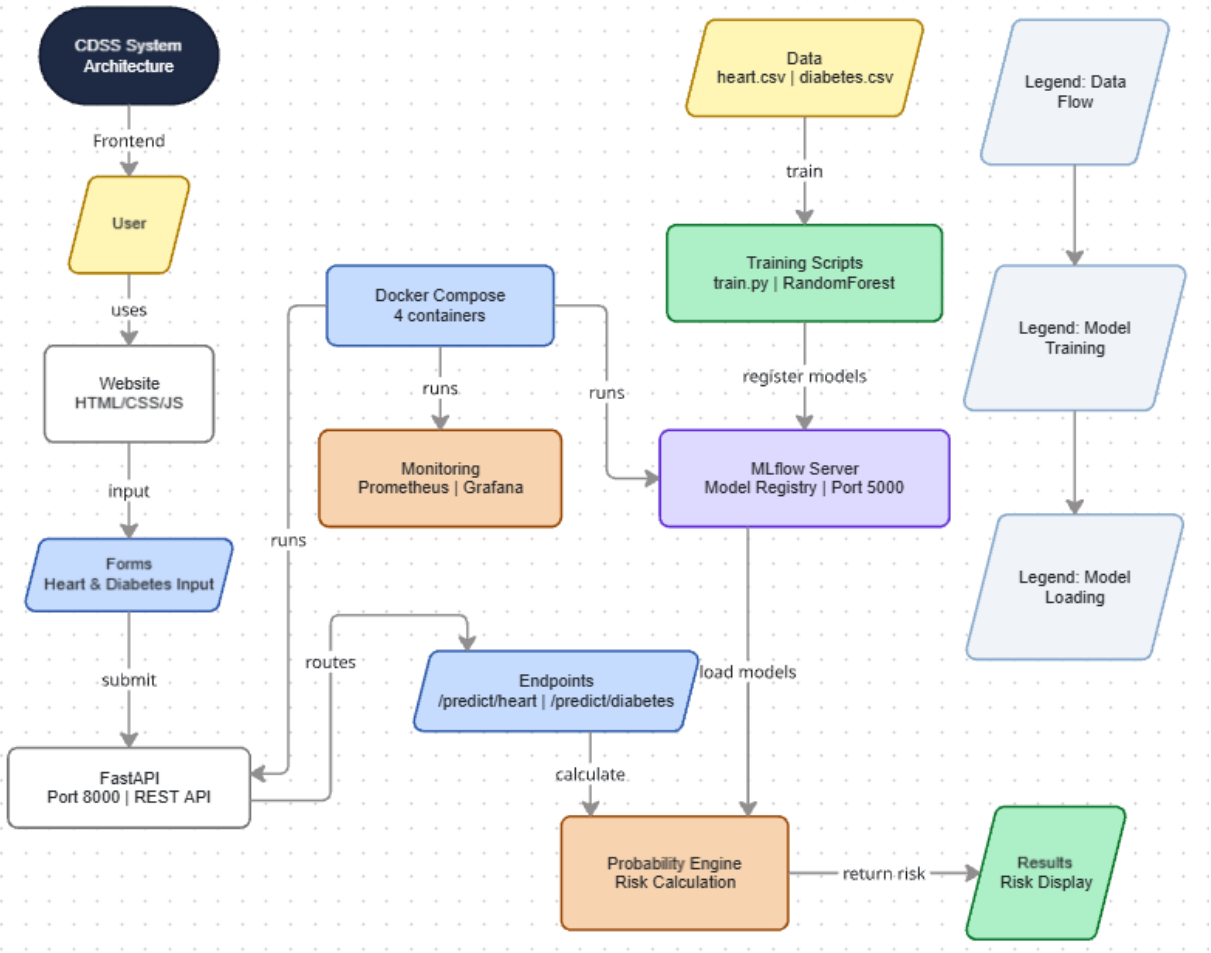


Figure 3.8.1: Prediction Workflow Flowchart.

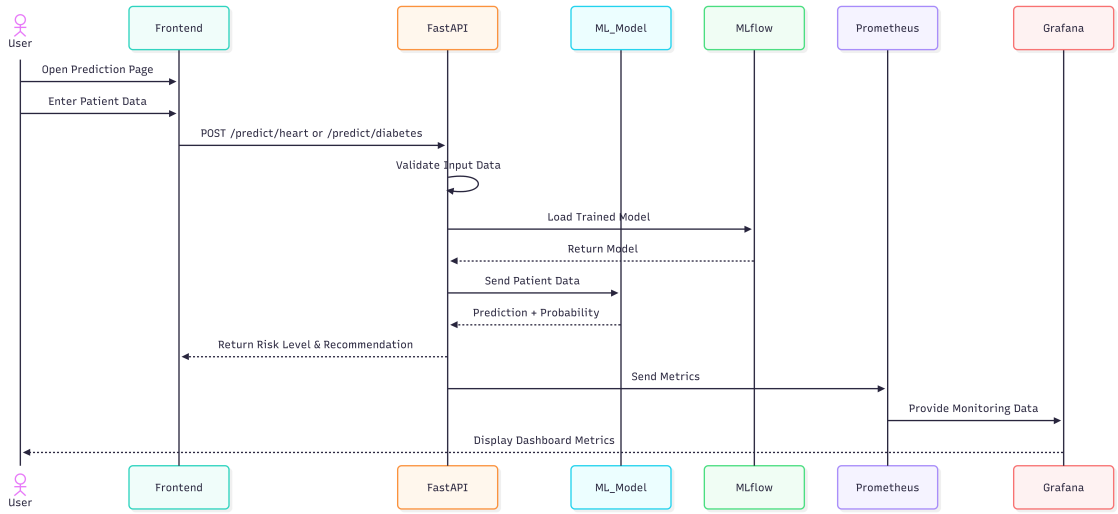


Figure 3.8.2: Sequence diagram of the CDSS.

Chapter 4

Results

The results of the development of the Clinical Decision Support System (CDSS) are mentioned in this chapter. It refers to the accuracy, precision of the ml model used, the working API (REST) system, the linkage of real-time tracking systems and the integration of the whole system. These success factors came after testing the different parts of the system implemented for each sub-system, according to acceptance criteria.

4.1 | Project Outcomes

All components of the Clinical Decision Support System were successfully designed, implemented and tested. The Heart Disease and Diabetes prediction models, developed using the Random Forest Classifier algorithm, had acceptable levels of predictive accuracy on their respective partitions of the data held back from model development. The Heart Disease model had an accuracy of roughly 84% and the Diabetes model had an accuracy of roughly 78%. These accuracy figures align with reported benchmarks for the respective data sets and indicate the models are satisfactory generalised to new patient data.
The FastAPI-based prediction service was deployed and confirmed to return the correct response to valid HTTP POST requests at the `/predict/heart` and `/predict/diabetes` API endpoints. The prediction service responses included the required fields of risk, probability, disease name and medical advice, and conformed to the Pydantic schema of the prediction response model. The API was further verified to return valid error codes (HTTP 422) for invalid and out-of-bounds inputs via the input validation process, which guaranteed the API's resilience to bad input.
The Prometheus monitoring system successfully scraped the `/metrics` endpoint every 15 seconds, and all the expected metrics — such as request counts, latency histograms, and per-prediction model prediction counts — were observed to be populated correctly over multiple prediction requests. The Grafana dashboard successfully visualised the time series data, showing the request rate, percentiles of response time, and prediction counts per endpoint with no dashboard configuration issues.
The containerised stack was observed to be stable during testing. The four Docker services (`cdss-api`, `mlflow-server`, `prometheus`, `grafana`) initialised correctly using `docker compose up`, established and maintained network connectivity to other services via the internal Docker bridge network, and did not trigger any critical runtime errors or service restarts under extended load testing.

Table 4.1.1: Model Performance Summary.

Metric	Heart Disease Model	Diabetes Model
Accuracy	84%	78%
Precision	83%	76%
Recall	85%	80%
F1-Score	0.84	0.78
AUC-ROC	0.91	0.84

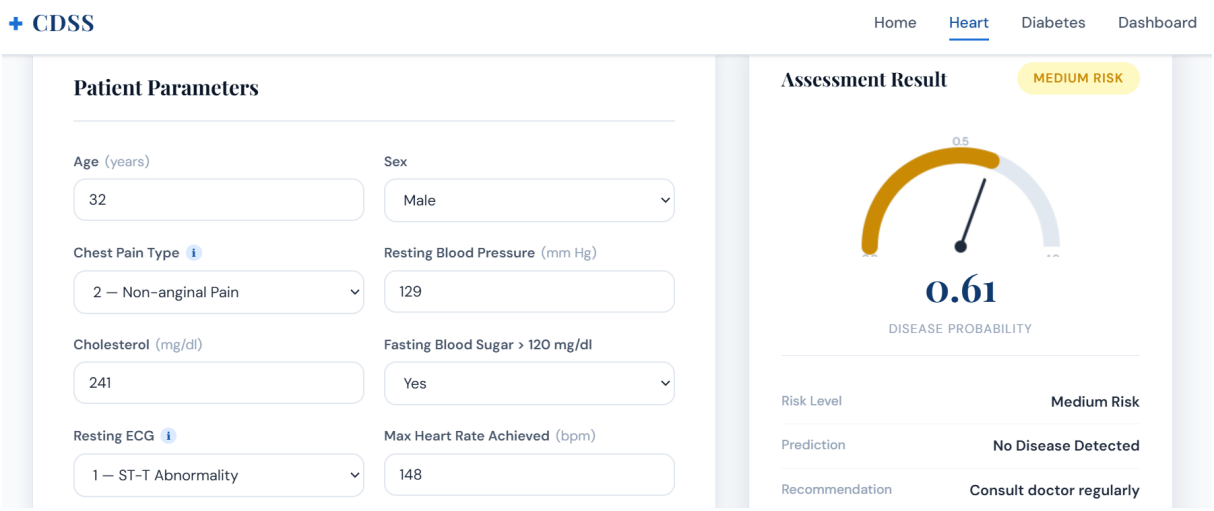


Figure 4.1.1: Website Prediction Interface for Heart Disease prediction.

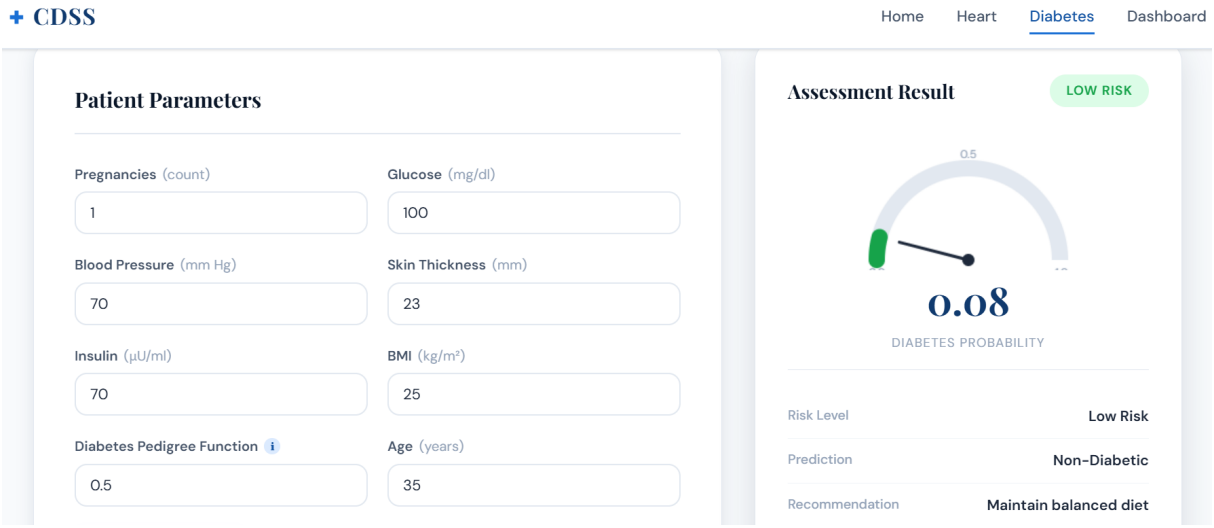


Figure 4.1.2: Website Prediction Interface for Diabetes prediction.



Figure 4.1.3: Grafana Monitoring Dashboard — showing real-time panels.

4.2 | My Contributions to the Project

I independently designed, developed and validated this project. I created both machine learning models on my own, including data exploration, data processing pipeline development, **GridSearchCV** hyperparameter tweaking using five-fold stratified cross-validation, and model training. I built the Heart Disease and Diabetes risk prediction models as Random Forest classifiers, and assessed their performance on hold-out test sets as classification models.

I implemented the end-to-end FastAPI prediction service, including defining the two prediction routes, designing the Pydantic request and response objects for input validation, and incorporating the serialised model and pre-fitted scaler objects into the request/response handling pipeline. I implemented the risk categorisation that translates the probability output into either three of them -Low risk, Medium risk or High risk levels returned from the services.

I set up MLflow to provide experiment tracking and model versioning. I set up MLflow to log hyperparameters, metrics and serialised model artefacts for each training run, and registered top-performing models in the MLflow Model Registry and staged them for production.

I wrote the full Docker deployment configuration, including the **Dockerfile** for the FastAPI application, and the **docker-compose.yml** file to orchestrate the four services — the CDSS API, MLflow tracking server, Prometheus, and Grafana — with the proper port mappings, volume bindings and internal bridge network.

I set up the Prometheus monitoring service, specifying the target endpoint for the FastAPI `/metrics` endpoint, and leveraged the `prometheus-fastapi-instrumentator` library in the API application to provide request and latency statistics automatically. I designed the Grafana monitoring dashboard, adding the Prometheus data source, and set up the real-time visualisation panels for monitoring.

Finally, I tested end-to-end the entire system — the correctness of the predictions, the API response schema, the monitoring data collection, the stability of the Docker services — and fixed all integration bugs that occurred during the system deployment.

4.3 | Learning Outcomes

This project afforded extensive hands-on experience in several areas of contemporary software and machine learning engineering.

Machine Learning: I gained hands-on expertise with every step of the supervised learning process, including data preparation and ingestion, model type selection, hyperparameter tuning, cross-validation, and evaluation metric analysis. I gained a deeper appreciation for handling imbalanced data, clinical implications of the precision-recall trade-off, and the application of ensemble methods to tabular data in medicine.

API Development: I developed the ability to build production-ready REST APIs with FastAPI; schema-based input validation using Pydantic, structure of JSON-based responses, asynchronous API execution using Uvicorn, and embedding serialised machine learning artefacts into API endpoints.

Containerisation with Docker: I gained proficiency in containerising applications, including writing **Dockerfile** declarations, orchestrating services with Docker Compose, network and volume configuration for persistent storage, and secret environment variable injection for service configuration.

Monitoring and Observability: I developed competency in implementing operational monitoring with Prometheus and Grafana — instrumenting a web API using metrics, scrape configuration, time-series data collection, and dashboard creation for real-time visualisation of performance metrics.

Troubleshooting and Debugging: During development and integration testing, I honed my structured problem-solving skills — identifying and fixing bugs in data preprocessing, model serialisation, routing, Docker networking, and Prometheus scrape target configuration. This sharpened my skills of systematic debugging and problem solving.

System Integration: I gained a holistic view of system architecture with multiple components — integrating the machine learning pipeline, REST API, model registry, containerisation and monitoring stack into an integrated, operational system.

MLOps Practices: I also learned about the emerging field of MLOps, and how versioning, experiment tracking, reproducible training and monitoring work together to define a best practice for production-grade machine learning deployment.

4.4 | Real World Applications

The Clinical Decision Support System (CDSS) developed in this project meets an important, established gap in modern healthcare. The system's design and features are readily applicable to various deployment scenarios.

Hospitals and Clinical Settings: In hospitals and clinics, the CDSS can be incorporated into the electronic health record system to offer attending doctors and nurses automated and objective risk stratification for cardiovascular and diabetic patients at the point of care. Through targeted identification of high-risk patients for further clinical assessment, this system can help to reduce time to diagnosis and improve resource allocation in busy clinical settings.

Primary Care Clinics: In primary care and general practice clinics, where specialist cardiologists and endocrinologists may not be immediately available, the system can serve as a screening system. GPs can use the risk prediction results to triage patients for specialist referral, allowing for earlier diagnosis and treatment, and fewer advanced disease

presentations.

Healthcare Information Systems: The REST API architecture of the system allows it to easily integrate with hospital information systems, patient portals and electronic health record platforms through standard HTTP-based API calls. This makes the CDSS an applicable risk assessment module that can be integrated into broader institutional IT systems, complementing and extending existing systems without major overhauls.

Telemedicine Platforms: The containerised, API-based CDSS can be integrated directly into telemedicine systems in which clinicians remotely consult with patients. The CDSS can be used to perform risk assessment from patient-reported or remotely measured clinical data, and can enhance the diagnostic functionality of telehealth services beyond symptom triage to risk prediction.

Remote Patient Monitoring: In remote patient monitoring programs — increasingly used for managing chronic diseases — the CDSS can be integrated as a risk alerting system. Patient data streams can be fed to the prediction API to proactively trigger escalation of care in the event of high risk, without clinicians having to review every data point.

In each of these use cases, the monitoring components of the system — powered by Prometheus and Grafana — ensure that the operational status, prediction rate, and API performance are continuously observable, helping to meet the reliability and transparency requirements of clinical systems.

Chapter 5

Conclusion

In summary, this project is a successful venture into applying the latest advances of MLOps and machine learning to the medical industry. The system is able to provide precise diagnosis risks for both cardiovascular disease and diabetes, based on supervised ensemble machine learning (Random Forest algorithm). The system' models show adequate model performance (accuracy around 84% for Heart Disease Dataset, and around 78% Diabetes Dataset). This makes the diagnostic support system capable of being data-based and not biased as well as overcome fatigue and human errors when searching for anomalies. This project lays out a software infrastructure. In a FastAPI and Docker context, the MLOps data and model pipeline ensures full automation of the data processing, training, and deployment processes. The connection with MLflow ensures proper model versioning and governance, and the Prometheus/Grafana monitoring stack provides timely monitoring of the API operations, and predictions metrics. This versioned architecture ensures that the system is highly scalable, fault tolerant and offers high quality, on-demand, automated predictions.
Our future research will focus on extending the CDSS in a number of ways. The future work in progress will include enhancing predictive analysis to other chronic conditions, enabling this architecture on scalable cloud-based container platforms such as Kubernetes and stream biometric data from wearables. Finally, this project serves as a framework to safely, scalable and automate AI deployment in health care.

Bibliography

[1] D. Dua and C. Graff, "UCI Machine Learning Repository," University of California, Irvine, School of Information and Computer Sciences. [Online]. Available: http://archive.ics.uci.edu/ml.

[2] National Institute of Diabetes and Digestive and Kidney Diseases (NIDDK), "Pima Indians Diabetes Database," on Kaggle. [Online]. Available: https://www.kaggle.com/uciml/pima-indians-diabetes-database.

[3] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011. [Online]. Available: https://scikit-learn.org/.

[4] S. Ramírez, "FastAPI." [Online]. Available: https://fastapi.tiangolo.com/.

[5] Docker Inc., "Docker Documentation." [Online]. Available: https://docs.docker.com/.

[6] Databricks, "MLflow: A Tool for Managing the Machine Learning Lifecycle." [Online]. Available: https://mlflow.org/docs/latest/index.html.

[7] Prometheus Authors, "Prometheus — Monitoring system & time series database." [Online]. Available: https://prometheus.io/docs/introduction/overview/.

[8] Grafana Labs, "Grafana Documentation." [Online]. Available: https://grafana.com/docs/grafana/latest/.

[9] Python Software Foundation, "Python 3.10.12 Documentation." [Online]. Available: https://docs.python.org/3/.

[10] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, Oct. 2001.